

Assignment 1: Implementation of a B+ Tree Index

Tamim Alam | talam@nmsu.edu | New Mexico State University

1. Overview

The program implements an in-memory B+ tree index with integer keys and integer pointers. It is started as `bplus init <d> <commands.txt>`: it builds an empty tree whose node capacity is set by d , executes every command from standard input in order, and exits. Nothing is written to disk and no state is kept between runs. The algorithms follow Chapter 10 of *Ramakrishnan and Gehrke*, where d is the order of the tree. The code has three parts: the **Node** structure, the **BPlusTree** class (the algorithms and the statistics) and the **command layer** (argument checking, parsing and the exact output messages).

2. Data Structures: Node Representation

Every node is a heap-allocated Node object. Nodes refer to each other with ordinary C++ pointers.

Node field	Meaning
isLeaf	node type
Keys	sorted keys (a vector); at most $2d$ per node
Pointers	leaf only: the record pointer of each key, <code>pointers[i]</code> belongs to <code>keys[i]</code>
Children	internal only: <code>keys.size() + 1</code> child node pointers, at most $2d + 1$
next, prev	leaf only: the neighbouring leaves; the leaf level forms a doubly linked list

Capacity: The parameter d sets how big a node can get. A leaf can hold up to $2d$ key-pointer pairs, and an internal node up to $2d$ keys and $2d + 1$ children. Every node except the root has to stay at least half full, which works out to at least d keys for both kinds of node, so an internal node always has at least $d + 1$ children. During an insertion, a node can briefly hold $2d + 1$ keys, but only right before it gets split. The tree object keeps track of the root pointer, d , the height, the number of nodes and the number of keys. These counts are updated whenever a split, a merge or a root change happens, so `PRINT STATISTICS` can report them without walking the tree. d has to be a whole number of at least 1; if `init` isn't given a valid d , the program prints a usage message on standard error and exits.

Memory Management: Each split creates one new node. When two nodes merge, the right node, now empty, is freed. When the root collapses, the old root is freed. When the program ends, the destructor frees the whole tree. I also built the program with `AddressSanitizer` and `UndefinedBehaviorSanitizer`, and neither reported any memory errors or leaks across the tests described below.

3. Overall Design and Algorithms

- Search:** Starting at the root, the search follows child i where $\text{keys}[i-1] \leq K < \text{keys}[i]$, found with a binary search (upper bound), until it reaches a leaf, and then does a binary search inside that leaf. Search returns the pointer, or -1 if the key isn't there. Since -1 is also a valid pointer value, the command layer uses an internal lookup that reports "found" separately, so a key stored with pointer -1 is still printed as found.
- Insert:** Insertion works recursively on the way down and doesn't use parent pointers. If the key already exists, the insert is rejected and the tree stays as it was. When a leaf overflows ($2d + 1$ entries), the first d entries stay put, the other $d + 1$ move to a new leaf, the new leaf's smallest key is copied up to the parent, and the new leaf is linked into the leaf chain. When an internal node overflows ($2d + 1$ keys), the first d keys stay, the middle key is pushed up, and the last d keys move to a new node with their $d + 1$ children. If the root splits, a new root is created and the tree grows by one level. Insertion never redistributes entries.
- Delete:** Deletion also works recursively. The key is removed from its leaf. If a node drops below d keys, it reports that to its parent, and the parent checks one sibling, either the left one, or the right one if the node is the leftmost child. If that sibling has more than d keys, the entries of the two nodes are split evenly between them and the parent's key is updated. For leaves, it becomes the new lowest key of the right node and for internal nodes, entries rotate through the parent. Otherwise, the right node of the pair is merged into the left one (for internal nodes, the parent's key is pulled down into the merged node). The right node is then freed and its entry removed from the parent, which may in turn become underfull itself. If an internal root ends up with no keys, it is discarded and its only child becomes the new root, so the tree loses one level. A delete that doesn't cause an underflow leaves the separator keys alone, so `PRINT` may show a key that is no longer in any leaf and searches still find the right leaf.
- Range Search:** The search first goes down to the leaf where k_1 would be. From there, it follows the links to the next leaves. It stops when it finds a key larger than k_2 . If no keys fall in the range, the result is empty. This is the "null" case, and the program prints the "no records" message.

- **Print and Statistics:** PRINT goes through the tree one level at a time, from top to bottom. It prints one line for each level. Each node appears inside [...]. Keys in internal nodes are shown as (k). Entries in leaves are shown as (k: p). PRINT STATISTICS prints three numbers: the height of the tree, the number of nodes and the number of keys.

Output Conventions: All messages match the examples in the assignment exactly. This includes the line "10 found, point is 201340". The init command prints nothing, so the output contains only the results of the commands. PRINT STATISTICS prints the same three lines as the example. The revised requirements no longer ask for disk-read and disk-write statistics, so those lines are left out. The height counts the levels of the tree. The statistics example shows height 2 with 4 nodes. A tree with three levels needs at least 7 nodes. So the example tree must have two levels. If height counted edges, that tree would have height 1 instead. The PRINT example agrees with this, since it numbers its levels 0 and 1. For this reason, an empty tree has height 1 and prints Level 0: []. If an input line is malformed, the program reports it on standard error and skips it.

4. Complexity

Let N be the number of keys. Every internal node except the root has between $d + 1$ and $2d + 1$ children. Because of this, the height of the tree is $O(\log N / \log d)$. A search visits one node on each level. At each node, it runs a binary search, which costs $O(\log d)$. Insert and delete also visit one node on each level on the way down. Sometimes a split, redistribution or merge moves up the tree. In that case, they do $O(d)$ work on each level on the way back up. A range search first goes down the tree. Then it takes one more step for each leaf in the range. PRINT visits every node in the tree. PRINT STATISTICS takes $O(1)$ time.

5. Testing

The program was tested locally on a Windows machine using VS Code and MSYS2 g++ 16.2. I first verified that the program compiled successfully and followed the required execution format using the init <d> parameter. I tested the sample commands provided in the assignment, including INSERT, SEARCH, DELETE, RANGESEARCH, PRINT, and PRINT STATISTICS, and verified that the output format matched the expected examples. I created additional test cases to verify different B+ tree operations. The tests included inserting multiple keys to force leaf and internal node splits, searching for existing and non-existing keys, performing range searches across multiple leaf nodes, and deleting keys to test redistribution, merging, and root shrinking. I tested the implementation using different values of d , including $d = 1$ and $d = 2$, to verify that the node capacity rules were correctly applied. The printed tree structures and statistics were manually checked after each operation to ensure that keys remained sorted, the tree remained balanced, and the number of keys and nodes were updated correctly. All tested cases completed successfully, and the program produced the expected results for the tested operations.

6. Use of Generative AI

I used generative AI tools as a learning and development assistant throughout this assignment. I first used AI to help me understand the assignment requirements, clarify the B+ tree concepts, and plan the implementation approach before writing the code. I did not ask for a complete solution. I used it step by step to understand the design choices and verify my implementation. AI helped with individual parts of the implementation, such as understanding the node structure, designing test cases, checking expected outputs, and reviewing the program behavior. I made the final decisions about the programming language, program structure, and implementation approach. After receiving suggestions from AI, I reviewed the changes, tested the program locally using VS Code, and verified that the output matched the assignment requirements.

I also used AI to help review my README and Design Document for clarity and completeness. However, I checked the content myself and modified it to accurately represent my implementation and testing process.

Example Prompts: "Please help me understand the assignment requirements and explain what needs to be implemented in plain language step by step." "Can you explain how a B+ tree insertion, splitting, deletion, and merging process should work?" "Please help me implement this part of the B+ tree step by step and explain what each function is doing." "Create test cases that can verify insertion, searching, splitting, deletion, redistribution, and merging in a B+ tree." "I have attached the output, could you please verify?"

The generative AI tools used for this assignment:

ChatGPT (OpenAI) - Used for learning assistance, implementation guidance, writing and improving codes, testing, verifying codes in another fresh session, and document drafting.

References:

- OpenAI ChatGPT: <https://chat.openai.com/>